- CHAPTER 1
  - overview
    - 1.1basic elements

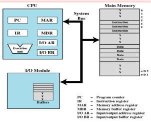
      
      CPU
      PC
      MAR
      IR
      MBR
      I/O AR
      I/O BR
      Execution unit
      Systems Bus
      Main Memory
      Instruction
      Data
      I/O Module
      Buffers
      PC = Program counter
      IR = Instruction register
      MAR = Memory address register
      MBR = Memory buffer register
      I/O AR = Input/output address register
      I/O BR = Input/output buffer register

      Figure 1.1 Computer Components: Top-Level View
      - processor（cpu）
        - include：PC，IR，MAR，MBR，I/O AR。。。
          - Memory address register (MAR)
            - Specifies the address for the next read or write
          - Memory buffer register (MBR)
            - Contains data written into memory or receives data read from memory
        - control the operation of the computer and performs its data processing functions
      - main memory
        - store data and programs
        - volatile
      - I/O modules
        - move data between computer and its external environment
      - System bus
        - provide for communication among processor，main memory and io modules
    - 1.2evolution of the microprocessor
    - 1.3Instruction execution

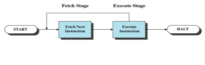
      
      Fetch Stage
      Execute Stage
      START
      Fetch Next Instruction
      Execute Instruction
      HALT

      Figure 1.2 Basic Instruction Cycle
      - the PC holds the address of the next instruction to be fetched
      - the fetched instruction is loaded into the IR

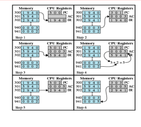
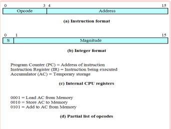
      
      Memory
      CPU Registers
      PC
      AC
      IR
      Step 1
      Step 2
      Step 3
      Step 4
      Step 5
      Step 6
      3 + 2 = 5

      Figure 1.4 Example of Program Execution (contents of memory and registers in hexadecimal)

      
      Opcode
      Address
      Magnitude
      S

      (a) Instruction format

      (b) Integer format

      Program Counter (PC) = Address of instruction
      Instruction Register (IR) = Instruction being executed
      Accumulator (AC) = Temporary storage

      (c) Internal CPU registers

      0001 = Load AC from Memory
      0010 = Store AC to Memory
      0101 = Add to AC from Memory

      (d) Partial list of opcodes

      Figure 1.3 Characteristics of a Hypothetical Machine

      a
      - Instruction Register
        - Fetched instruction is placed in the instruction register
    - 1.4interrupts
      - reason
        - Most I/O devices are slower than the processor
        - Processor must pause to wait for device
        - Interrupt the normal sequencing of the processor
        - Interrupts are provided primarily as a way to improve processor utilization
      - classes of interrupts
        - program
          - arithmetic overflow ,division by zero....
        - Timer
          - generated by a timer within the processor
        - I/O
          - signal normal completion of an operation
        - Hardware failure
          - power failure , memory parity error....

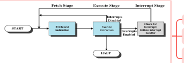
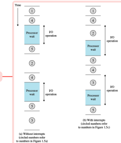
      
      Fetch Stage
      Execute Stage
      Interrupt Stage
      START
      Fetch next instruction
      Execute instruction
      Interrupts Disabled
      Interrupts Enabled
      Check for interrupt; initiate interrupt handler
      HALT

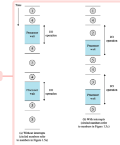
      Figure 1.7 Instruction Cycle with Interrupts
        - Processor checks for interrupts
        - If no interrupts fetch the next instruction for the current program
        - If an interrupt is pending, suspend execution of the current program, and execute the interrupt-handler routine
      - Timing Diagram Based on long I/O Wait

        
        Time
        Processor wait
        I/O operation

        (a) Without interrupts (circled numbers refer to numbers in Figure 1.5a)

        (b) With interrupts (circled numbers refer to numbers in Figure 1.5c)

        Figure 1.9 Program Timing: Long I/O Wait
      - Timing Diagram Based on Short I/O Wait

        
        Time
        Processor wait
        I/O operation

        (a) Without interrupts (circled numbers refer to numbers in Figure 1.5a)

        (b) With interrupts (circled numbers refer to numbers in Figure 1.5c)

        Figure 1.9 Program Timing: Long I/O Wait
      - Simple Interrupt Processing

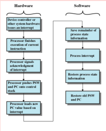
        
        Hardware
        Software
        Device controller or other system hardware issues an interrupt
        Processor finishes execution of current instruction
        Processor signals acknowledgment of interrupt
        Processor pushes PSW and PC onto control stack
        Processor loads new PC value based on interrupt
        Save remainder of process state information
        Process interrupt
        Restore process state information
        Restore old PSW and PC

        Figure 1.10 Simple Interrupt Processing
      - multiple interrupts
        - disable interrupt while an interrupt is being processed
        - define priorities for interrupts and to allow an interrupt of higher to case a lower-priority interrupt handler to be interrupted
    - 1.5the memory hierarchy
      - Memory Hierarchy (存储层次)
        - Faster access time, greater cost per bit
        - Greater capacity, smaller cost per bit
        - Greater capacity, slower access speed
      - Going Down the Hierarchy
        - decreasing cost per bit
        - increase capacity
        - increase access time
        - decreasing frequency of access to the memory by the processor

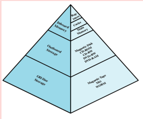
      
      Registers
      Cache
      Main Memory
      Inboard Memory
      Outboard Storage
      Magnetic Disk
      CD-ROM
      CD-RW
      DVD-RW
      DVD-RAM
      Off-line Storage
      Magnetic Tape
      MO
      WORM

      Figure 1.14 The Memory Hierarchy
      - second memory
        - Nonvolatile (非易失的)
        - store program and data files
        - visible to the programmer only in terms of files and records
        - Auxiliary memory (辅助存储)
    - 1.6 cache memory

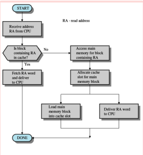
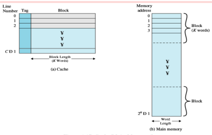
      
      START
      RA - read address
      Receive address RA from CPU
      Is block containing RA in cache?
      No
      Yes
      Access main memory for block containing RA
      Allocate cache slot for main memory block
      Fetch RA word and deliver to CPU
      Load main memory block into cache slot
      Deliver RA word to CPU
      DONE

      Figure 1.18 Cache Read Operation

      
      Line Number
      Tag
      Block
      Memory address
      Block Length (K Words)
      Block (K words)
      Word Length

      (a) Cache

      (b) Main memory

      Figure 1.17 Cache/Main-Memory Structure
      - Invisible to operating system
      - Increase the speed of memory
      - Processor speed is faster than memory speed
      - Exploit the principle of locality
        - 利用局部性原理
    - Processor Registers(寄存器)
      - Memory address register (MAR)
        - Specifies the address for the next read or write
      - Memory buffer register (MBR)
        - Contains data written into memory or receives data read from memory
      - I/O address register
      - I/O buffer register
      - User-visible registers
        - Enable programmer to minimize main-memory references by optimizing register use

---

- File management system consists of system utility programs that run as privileged applications
- Input to applications is by means of a file
- Output is saved in a file for long-term storage
- Sharable between processes
- File Operations
  - Create
  - Delete
  - Open
  - Close
  - Read
  - Write
- Objectives for a File Systems
  - Meet the data management needs and requirements of the user
  - Guarantee that the data in the file are valid
  - Optimize performance
  - Provide I/O support for a variety of storage device types
  - Minimize or eliminate the potential for lost or destroyed data
  - Provide a standardized set of I/O interface routines
  - Provide I/O support for multiple users
- Minimal Set of Requirements
  - Each user should be able to create, delete, read, write and modify files
  - Each user may have controlled access to other users' files
  - Each user may control what type of accesses are allowed to the users' files
  - Each user should be able to restructure the user's files in a form appropriate to the problem
  - Each user should be able to move data between files
  - Each user should be able to back up and recover the user's files in case of damage
  - Each user should be able to access the user's files by using symbolic names
- Device Drivers
  - Lowest level
  - Communicates directly with peripheral devices
  - Responsible for starting I/O operations on a device
  - Processes the completion of an I/O request
- Basic File System
  - Physical I/O
  - Deals with exchanging blocks of data
  - Concerned with the placement of blocks
  - Concerned with buffering blocks in main memory
- Basic I/O Supervisor
  - Responsible for file I/O initiation and termination
  - Control structures are maintained
  - Concerned with selection of the device on which file I/O is to be performed
  - Concerned with scheduling access to optimize performance
- Logical I/O
  - Enables users and applications to access file as a byte stream
  - Maintains basic data about file

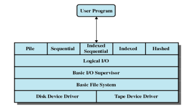

User Program
Pile
Sequential
Indexed Sequential
Indexed
Hashed
Logical I/O
Basic I/O Supervisor
Basic File System
Disk Device Driver
Tape Device Driver

Figure 12.1 File System Software Architecture

- File Management Functions
  - Identify and locate a selected file
  - Use a directory to describe the location of all files plus their attributes
  - On a shared system describe user access control
  - Blocking for access to files
  - Allocate files to free blocks
  - Manage free storage for available blocks

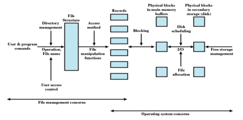
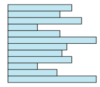

Directory management
File Structure
Records
Physical blocks in main memory buffers
Physical blocks in secondary storage (disk)
User & program comands
Operation, File name
Access method
File manipulation functions
Blocking
Disk scheduling
I/O
Free storage management
File allocation
User access control
File management concerns
Operating system concerns

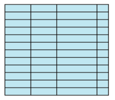
Figure 12.2 Elements of File Management

- File Organization
  - Pile
    - Data are collected in the order they arrive
    - Purpose is to accumulate a mass of data and save it
    - Records may have different fields
    - No structure
    - Record access is by exhaustive search

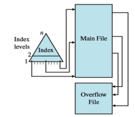
    Variable-length records
    Variable set of fields
    Chronological order

    (a) Pile File
  - Sequential File
    - Fixed format used for records
    - Records are the same length
    - All fields the same (order and length)
    - Field names and lengths are attributes of the file
    - One field is the key filed
      - Uniquely identifies the record
      - Records are stored in key sequence
    - New records are placed in a log file or transaction file
    - Batch update is performed to merge the log file with the master file

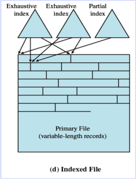
    Fixed-length records
    Fixed set of fields in fixed order
    Sequential order based on key field

    (b) Sequential File
  - Indexed Sequential File
    - Index provides a lookup capability to quickly reach the vicinity of the desired record
      - Contains key field and a pointer to the main file
      - Indexed is searched to find highest key value that is equal to or precedes the desired key value
      - Search continues in the main file at the location indicated by the pointer
    - New records are added to an overflow file
    - Record in main file that precedes it is updated to contain a pointer to the new record
    - The overflow is merged with the main file during a batch update
    - Multiple indexes for the same key field can be set up to increase efficiency

    
    Index levels
    n
    2
    1
    Index
    Main File
    Overflow File

(c) Indexed Sequential File

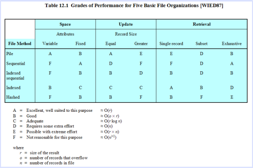
- CHAPTER 12
  - File Management
    - File Management System
      - Indexed File
        - Uses multiple indexes for different key fields
        - May contain an exhaustive index that contains one entry for every record in the main file
        - May contain a partial index


Exhaustive index
Exhaustive index
Partial index
Primary File
(variable-length records)

(d) Indexed File

      - Hashed File
        - Directly access a block at a known address
        - Key field required for each record

Table 12.1 Grades of Performance for Five Basic File Organizations [WIED87]



| | Space | | Update | | Retrieval | | |
|---|---|---|---|---|---|---|---|
| | Attributes | | Record Size | | | | |
| File Method | Variable | Fixed | Equal | Greater | Single record | Subset | Exhaustive |
| Pile | A | B | A | E | E | D | B |
| Sequential | F | A | D | F | F | D | A |
| Indexed sequential | F | B | B | D | B | D | B |
| Indexed | B | C | C | C | A | B | D |
| Hashed | F | B | B | F | B | F | E |

A = Excellent, well suited to this purpose $\approx O(r)$
B = Good $\approx O(o \times r)$
C = Adequate $\approx O(r \log n)$
D = Requires some extra effort $\approx O(n)$
E = Possible with extreme effort $\approx O(r \times n)$
F = Not reasonable for this purpose $\approx O(n^{>1})$

where
$r$ = size of the result
$o$ = number of records that overflow
$n$ = number of records in file

      - File Directories
        - Contains information about files
          - Attributes
          - Location
          - Ownership
        - Directory itself is a file owned by the operating system
        - Provides mapping between file names and the files themselves
        - Simple Structure for a Directory
          - List of entries, one for each file
          - Sequential file with the name of the file serving as the key
          - Provides no help in organizing the files
          - Forces user to be careful not to use the same name for two different files
        - Two-level Scheme for a Directory
          - One directory for each user and a master directory
          - Master directory contains entry for each user
            - Provides address and access control information
          - Each user directory is a simple list of files for that user
          - Still provides no help in structuring collections of files
        - Hierarchical, or Tree-Structured Directory
          - Master directory with user directories underneath it
          - Each user directory may have subdirectories and files as entries
        - Tree-Structured Directory

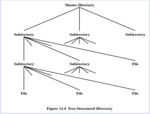
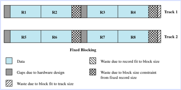

Master Directory
Subdirectory
Subdirectory
Subdirectory
Subdirectory
Subdirectory
File
File
File
File

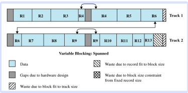
Figure 12.4 Tree-Structured Directory

        - Hierarchical, or Tree-Structured Directory
          - Files can be located by following a path from the root, or master, directory down various branches
            - This is the pathname for the file
          - Can have several files with the same file name as long as they have unique path names
          - Current directory is the working directory
          - Files are referenced relative to the working directory
      - File Sharing
        - In multiuser system, allow files to be shared among users
        - Two issues
          - Access rights
            - None
              - User may not know of the existence of the file
              - User is not allowed to read the user directory that includes the file
            - Knowledge
              - User can only determine that the file exists and who its owner is
            - Execution
              - The user can load and execute a program but cannot copy it
            - Reading
              - The user can read the file for any purpose, including copying and execution
            - Appending
              - The user can add data to the file but cannot modify or delete any of the file's contents
            - Updating
              - The user can modify, deleted, and add to the file's data. This includes creating the file, rewriting it, and removing all or part of the data
            - Changing protection
              - User can change access rights granted to other users
            - Deletion
              - User can delete the file
            - Owners
              - Has all rights previously listed
              - May grant rights to others using the following classes of users
          - Management of simultaneous access
      - Fixed Blocking


R1
R2
R3
R4
Track 1
R5
R6
R7
R8
Track 2
Data
Gaps due to hardware design
Waste due to block fit to track size
Waste due to record fit to block size
Waste due to block size constraint from fixed record size

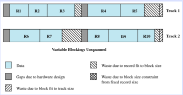
Fixed Blocking

      - Variable Blocking: Spanned


R1
R2
R3
R4
R4
R5
R6
Track 1
R6
R7
R8
R9
R9
R10
R11
R12
R13
Track 2
Data
Gaps due to hardware design
Waste due to block fit to track size
Waste due to record fit to block size
Waste due to block size constraint from fixed record size

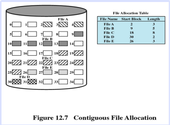
Variable Blocking: Spanned

      - Variable Blocking: Unspanned


R1
R2
R3
R4
R5
Track 1
R6
R7
R8
R9
R10
Track 2
Data
Gaps due to hardware design
Waste due to block fit to track size
Waste due to record fit to block size
Waste due to block size constraint from fixed record size

Variable Blocking: Unspanned

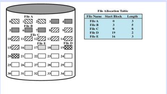
      - Space must be allocated to files
      - Must keep track of the space available for allocation
      - Preallocation(预先分配)
        - Need the maximum size for the file at the time of creation
        - Difficult to reliably estimate the maximum potential size of the file
        - Tend to overestimated file size so as not to run out of space
      - Contiguous Allocation(连续分配)
        - Single set of blocks is allocated to a file at the time of creation
        - Only a single entry in the file allocation table
          - Starting block and length of the file
        - External fragmentation will occur
          - Need to perform compaction


File A
File B
File C
File E
File D
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34

File Allocation Table

| File Name | Start Block | Length |
|---|---|---|
| File A | 2 | 3 |
| File B | 9 | 5 |
| File C | 18 | 8 |
| File D | 30 | 2 |
| File E | 26 | 3 |

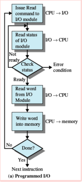
Figure 12.7 Contiguous File Allocation


File A
File B
File C
File E
File D
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34

File Allocation Table

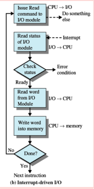
| File Name | Start Block | Length |
|---|---|---|
| File A | 0 | 3 |
| File B | 3 | 5 |
| File C | 8 | 8 |
| File D | 19 | 2 |
| File E | 16 | 3 |

- other
  - Multiprogramming (多道)
    - Processor has more than one program to execute
    - The sequence the programs are executed depend on their relative priority and whether they are waiting for I/O
    - After an interrupt handler completes, control may not return to the program that was executing at the time of the interrupt
  - Programmed I/O
    - I/O module performs the action, not the processor
    - Sets appropriate bits in the I/O status register
    - No interrupts occur
    - Processor checks status until operation is complete


Issue Read command to I/O module
CPU → I/O
Read status of I/O module
I/O → CPU
Not ready
Check status
Error condition
Ready
Read word from I/O Module
I/O → CPU
Write word into memory
CPU → memory
No
Done?
Yes
Next instruction

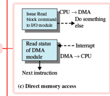
(a) Programmed I/O

  - Interrupt-Driven I/O
    - Processor is interrupted when I/O module ready to exchange data
    - Processor saves context of program executing and begins executing interrupt-handler
    - No needless waiting
    - Consumes a lot of processor time because every word read or written passes through the processor


Issue Read command to I/O module
CPU → I/O
Do something else
Read status of I/O module
Interrupt
I/O → CPU
Check status
Error condition
Ready
Read word from I/O Module
I/O → CPU
Write word into memory
CPU → memory
No
Done?
Yes
Next instruction

(b) Interrupt-driven I/O

  - Direct Memory Access (DMA)
    - I/O exchanges occur directly with memory
    - Processor grants I/O module authority to read from or write to memory
    - Relieves the processor responsibility for the exchange


Issue Read block command to I/O module
CPU → DMA
Do something else
Read status of DMA module
Interrupt
DMA → CPU
Next instruction

(c) Direct memory access

    - Transfers a block of data directly to or from memory
    - An interrupt is sent when the transfer is complete
    - Processor continues with other work

- CHAPTER 2
  - Operating System Overview
    - A program that controls the execution of application programs
    - An interface between applications and hardware

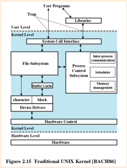

User Programs
Trap
Libraries
User Level
Kernel Level
System Call Interface
File Subsystem
Process Control Subsystem
Inter-process communication
Scheduler
Memory management
Buffer Cache
character
block
Device Drivers
Hardware Control
Kernel Level
Hardware Level
Hardware

Figure 2.15 Traditional UNIX Kernel [BACH86]

  - Operating System Objectives
    - Convenience
      - Makes the computer more convenient to use
    - Efficiency
      - Allows computer system resources to be used in an efficient manner
    - Ability to evolve
      - Permit effective development, testing, and introduction of new system functions without interfering with service
  - Layers of Computer System

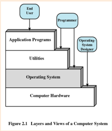
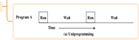

End User
Programmer
Operating-System Designer
Application Programs
Utilities
Operating System
Computer Hardware

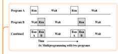
Figure 2.1 Layers and Views of a Computer System

  - Services Provided by the OS
    - Program development
    - Program execution
    - Access to I/O devices
    - Controlled access to files
    - System access
    - Error detection and response
    - Accounting
      - Collect usage statistics
      - Monitor performance
      - Used to anticipate future enhancements
      - Used for billing purposes
  - Kernel (核心/内核)
    - Portion of operating system that is in main memory
    - Contains most frequently used functions
    - Also called the nucleus (核心)
  - Evolution of an OS
    - Serial Processing
      - No operating system
      - Machines run from a console with display lights, toggle switches, input device, and printer
      - Schedule time
      - Setup included loading the compiler, source program, saving compiled program, and loading and linking
    - Simple Batch Systems
      - Monitors
        - Software that controls the sequence of events
        - Batch jobs together
        - Program branches back to monitor when finished
      - Job Control Language (JCL)
        - Special type of programming language
        - Provides instruction to the monitor
    - Uniprogramming (单任务)
      - Processor must wait for I/O instruction to complete before preceding

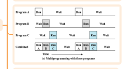


(a) Uniprogramming

    - Multiprogramming (多道/多任务)
      - When one job needs to wait for I/O, the processor can switch to the other job



(b) Multiprogramming with two programs



(c) Multiprogramming with three programs

      - Using multiprogramming to handle multiple interactive jobs
      - Processor's time is shared among multiple users
      - Multiple users simultaneously access the system through terminals
    - Time Sharing

Figure 12.8 Contiguous File Allocation (After Compaction)

- Secondary Storage Management
  - Chained Allocation
    - Allocation on basis of individual block
    - Each block contains a pointer to the next block in the chain
    - Only single entry in the file allocation table
    - Starting block and length of file
    - No external fragmentation
    - Best for sequential files
    - No accommodation of the principle of locality

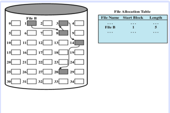

File B
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
File Allocation Table
File Name
Start Block
Length
File B
1
5

Figure 12.9 Chained Allocation

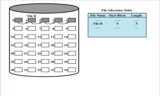

File B
File Allocation Table
File Name
Start Block
Length
File B
0
5

Figure 12.10 Chained Allocation (After Consolidation)

  - Indexed Allocation
    - File allocation table contains a separate one-level index for each file
    - The index has one entry for each portion allocated to the file
    - The file allocation table contains block number for the index

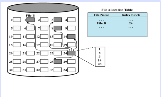

File B
File Allocation Table
File Name
Index Block
File B
24
1
8
3
14
28

Figure 12.11 Indexed Allocation with Block Portions

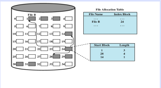

File B
File Allocation Table
File Name
Index Block
File B
24
Start Block
Length
1
28
14
3
4
1

Figure 12.12 Indexed Allocation with Variable-Length Portions

  - Inodes
    - Index node (索引节点)
    - Control structure that contains key information for a particular file

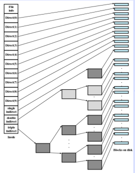
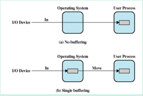

File info
Direct(0)
Direct(1)
Direct(2)
Direct(3)
Direct(4)
Direct(5)
Direct(6)
Direct(7)
Direct(8)
Direct(9)
single indirect
double indirect
triple indirect
Inode
Blocks on disk

Figure 12.13 Layout of a UNIX File on Disk

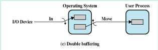
- I/O Management and Disk Scheduling
  - Categories of I/O Devices
    - Human readable (used to communicate with the user)
    - Machine readable(used to communicate with electronic equipment)
    - Communication(used to communicate with remote devices)
  - Three Techniques for Performing I/O
    - Programmed I/O
      - Process is busy-waiting for the operation to complete
    - Interrupt-driven I/O
      - I/O command is issued
      - Processor continues executing instructions
      - I/O module sends an interrupt when done
    - Direct Memory Access (DMA)
      - DMA module controls exchange of data between main memory and the I/O device
      - Processor interrupted only after entire block has been transferred
  - I/O Buffering
    - Reasons for buffering
      - Processes must wait for I/O to complete before proceeding
      - Certain pages must remain in main memory during I/O
    - Single Buffer
      - Operating system assigns a buffer in main memory for an I/O request


Operating System
User Process
I/O Device
In
Operating System
User Process
I/O Device
In
Move

(a) No buffering

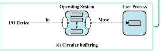
(b) Single buffering

      - Block-oriented
        - Input transfers made to buffer
        - Block moved to user space when needed
        - Another block is moved into the buffer
        - Read ahead
        - User process can process one block of data while next block is read in
        - Swapping can occur since input is taking place in system memory, not user memory
        - Operating system keeps track of assignment of system buffers to user processes
    - Double Buffer
      - Use two system buffers instead of one
      - A process can transfer data to or from one buffer while the operating system empties or fills the other buffer


Operating System
User Process
I/O Device
In
Move

(c) Double buffering

    - Circular Buffer
      - More than two buffers are used
      - Each individual buffer is one unit in a circular buffer
      - Used when I/O operation must keep up with process


Operating System
User Process
I/O Device
In
Move

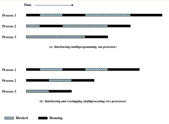
(d) Circular buffering

  - Seek time is the reason for differences in performance
  - First-in, first-out (FIFO)
    - Process request sequentially
    - Fair to all processes
    - Approaches random scheduling in performance if there are many processes

**CHAPTER 11**

---

- Modern Operating Systems
  - Microkernel architecture
    - Assigns only a few essential functions to the kernel
  - Distributed operating systems
    - Provides the illusion of a single main memory space and single secondary memory space
  - Object-oriented design
    - Used for adding modular extensions to a small kernel
    - Enables programmers to customize an operating system without disrupting system integrity
- Multiprogramming and Multiprocessing



(a) Interleaving (multiprogramming, one processor)

(b) Interleaving and overlapping (multiprocessing; two processors)

Figure 2.12 Multiprogramming and Multiprocessing

- Features of Modern Operating Systems
  - Symmetric multiprocessing (SMP)
    - There are multiple processors
    - These processors share same main memory and I/O facilities
    - All processors can perform the same functions
  - Multithreading
    - Process is divided into threads that can run concurrently
- Traditional UNIX Kernel

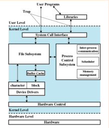

User Programs
Trap
Libraries
User Level
Kernel Level
System Call Interface
File Subsystem
Process Control Subsystem
Inter-process communication
Scheduler
Memory management
Buffer Cache
character
block
Device Drivers
Hardware Control
Kernel Level
Hardware Level
Hardware

Figure 2.15 Traditional UNIX Kernel [BACH86]

- Process (进程)
  - A program in execution
  - An instance of a program running on a computer
  - The entity that can be assigned to and executed on a processor
  - A unit of activity characterized by the execution of a sequence of instructions, a current state (当前状态), and an associated set of system instructions
- Process Elements
  - Identifier
  - State
  - Priority
  - Program counter
  - Memory pointers
  - Context data (上下文数据)
  - I/O status information
  - Accounting information
- Process Control Block (PCB)
  - Contains the process elements
  - Created and manage by the operating system
  - Allows support for multiple processes
  - Process Entry (进程表项)

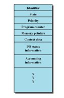

Identifier
State
Priority
Program counter
Memory pointers
Context data
I/O status information
Accounting information

Figure 3.1 Simplified Process Control Block

- Trace of Processes
  - the list of the sequence of instruction that execute for the process

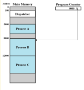
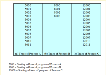

Address
Main Memory
Program Counter
8000
0
100
Dispatcher
5000
Process A
8000
Process B
12000
Process C

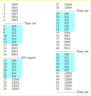
Figure 3.2 Snapshot of Example Execution (Figure 3.4) at Instruction Cycle 13



| (a) Trace of Process A | (b) Trace of Process B | (c) Trace of Process C |
|---|---|---|
| 5000 | 8000 | 12000 |
| 5001 | 8001 | 12001 |
| 5002 | 8002 | 12002 |
| 5003 | 8003 | 12003 |
| 5004 | | 12004 |
| 5005 | | 12005 |
| 5006 | | 12006 |
| 5007 | | 12007 |
| 5008 | | 12008 |
| 5009 | | 12009 |
| 5010 | | 12010 |
| 5011 | | 12011 |

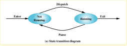
5000 = Starting address of program of Process A
8000 = Starting address of program of Process B
12000 = Starting address of program of Process C

Figure 3.3 Traces of Processes of Figure 3.2

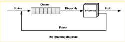


```
1   5000                 27  12004
2   5001                 28  12005
3   5002                 ----------- Time out
4   5003                 29  100
5   5004                 30  101
6   5005                 31  102
----------- Time out     32  103
7   100                  33  104
8   101                  34  105
9   102                  35  5006
10  103                  36  5007
11  104                  37  5008
12  105                  38  5009
13  8000                 39  5010
14  8001                 40  5011
15  8002                 ----------- Time out
16  8003                 41  100
----------- I/O request  42  101
17  100                  43  102
18  101                  44  103
19  102                  45  104
20  103                  46  105
21  104                  47  12006
22  105                  48  12007
23  12000                49  12008
24  12001                50  12009
25  12002                51  12010
26  12003                52  12011
                         ----------- Time out
```

- Two-State Process Model


Dispatch
Enter
Not Running
Running
Exit
Pause

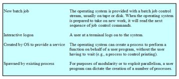
(a) State transition diagram


Queue
Enter
Dispatch
Processor
Exit
Pause

(b) Queuing diagram

- Process Creation (进程创建)

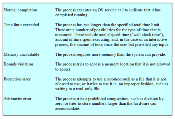
Table 3.1 Reasons for Process Creation



| | |
|---|---|
| New batch job | The operating system is provided with a batch job control stream, usually on tape or disk. When the operating system is prepared to take on new work, it will read the next sequence of job control commands. |
| Interactive logon | A user at a terminal logs on to the system. |
| Created by OS to provide a service | The operating system can create a process to perform a function on behalf of a user program, without the user having to wait (e.g., a process to control printing). |
| Spawned by existing process | For purposes of modularity or to exploit parallelism, a user program can dictate the creation of a number of processes. |

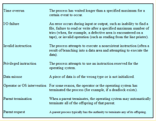
- Process Termination (进程终止)

Table 3.2 Reasons for Process Termination



| | |
|---|---|
| Normal completion | The process executes an OS service call to indicate that it has completed running. |
| Time limit exceeded | The process has run longer than the specified total time limit. There are a number of possibilities for the type of time that is measured. These include total elapsed time ("wall clock time"), amount of time spent executing, and, in the case of an interactive process, the amount of time since the user last provided any input. |
| Memory unavailable | The process requires more memory than the system can provide. |
| Bounds violation | The process tries to access a memory location that it is not allowed to access. |
| Protection error | The process attempts to use a resource such as a file that it is not allowed to use, or it tries to use it in an improper fashion, such as writing to a read-only file. |
| Arithmetic error | The process tries a prohibited computation, such as division by zero, or tries to store numbers larger than the hardware can accommodate. |

Table 3.2 Reasons for Process Termination



| | |
|---|---|
| Time overrun | The process has waited longer than a specified maximum for a certain event to occur. |
| I/O failure | An error occurs during input or output, such as inability to find a file, failure to read or write after a specified maximum number of tries (when, for example, a defective area is encountered on a tape), or invalid operation (such as reading from the line printer). |
| Invalid instruction | The process attempts to execute a nonexistent instruction (often a result of branching into a data area and attempting to execute the data). |
| Privileged instruction | The process attempts to use an instruction reserved for the operating system. |
| Data misuse | A piece of data is of the wrong type or is not initialized. |
| Operator or OS intervention | For some reason, the operator or the operating system has terminated the process (for example, if a deadlock exists). |
| Parent termination | When a parent terminates, the operating system may automatically terminate all of the offspring of that parent. |
| Parent request | A parent process typically has the authority to terminate any of its offspring. |

- Five-State Process Model

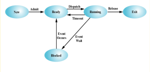
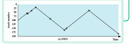
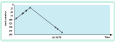

New
Admit
Ready
Dispatch
Running
Release
Exit
Timeout
Event Occurs
Event Wait
Blocked

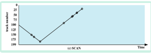
Figure 3.6 Five-State Process Model



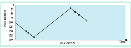
- Disk Scheduling Policies
  - Shortest Service Time First
    - Select the disk I/O request that requires the least movement of the disk arm from its current position
    - Always choose the minimum Seek time



  - SCAN
    - Disk arm moves in one direction only, satisfying all outstanding requests until it reaches the last track in that direction
    - Direction is reversed



  - C-SCAN (circular SCAN)
    - Restricts scanning to one direction only
    - When the last track has been visited in one direction, the arm is returned to the opposite end of the disk and the scan begins again



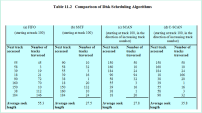
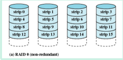


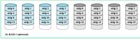
Table 11.2 Comparison of Disk Scheduling Algorithms

| (a) FIFO (starting at track 100) | | (b) SSTF (starting at track 100) | | (c) SCAN (starting at track 100, in the direction of increasing track number) | | (d) C-SCAN (starting at track 100, in the direction of increasing track number) | |
|---|---|---|---|---|---|---|---|
| Next track accessed | Number of tracks traversed | Next track accessed | Number of tracks traversed | Next track accessed | Number of tracks traversed | Next track accessed | Number of tracks traversed |
| 55 | 45 | 90 | 10 | 150 | 50 | 150 | 50 |
| 58 | 3 | 58 | 32 | 160 | 10 | 160 | 10 |
| 39 | 19 | 55 | 3 | 184 | 24 | 184 | 24 |
| 18 | 21 | 39 | 16 | 90 | 94 | 18 | 166 |
| 90 | 72 | 38 | 1 | 58 | 32 | 38 | 20 |
| 160 | 70 | 18 | 20 | 55 | 3 | 39 | 1 |
| 150 | 10 | 150 | 132 | 39 | 16 | 55 | 16 |
| 38 | 112 | 160 | 10 | 38 | 1 | 58 | 3 |
| 184 | 146 | 184 | 24 | 18 | 20 | 90 | 32 |
| Average seek length | 55.3 | Average seek length | 27.5 | Average seek length | 27.8 | Average seek length | 35.8 |

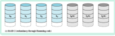
- RAID
  - Redundant Array of Independent Disks
  - Set of physical disk drives viewed by the operating system as a single logical drive
  - Data are distributed across the physical drives of an array
  - Redundant disk capacity is used to store parity information
  - RAID 0 (non-redundant)


strip 0
strip 4
strip 8
strip 12
strip 1
strip 5
strip 9
strip 13
strip 2
strip 6
strip 10
strip 14
strip 3
strip 7
strip 11
strip 15
(a) RAID 0 (non-redundant)

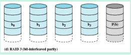
  - RAID 1 (mirrored)


(b) RAID 1 (mirrored)

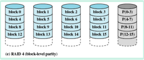
  - RAID 2 (redundancy through Hamming code)


(c) RAID 2 (redundancy through Hamming code)

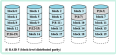
  - RAID 3 (bit-interleaved parity)


b0
b1
b2
b3
P(b)
(d) RAID 3 (bit-interleaved parity)

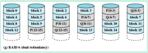
  - RAID 4 (block-level parity)


block 0
block 4
block 8
block 12
block 1
block 5
block 9
block 13
block 2
block 6
block 10
block 14
block 3
block 7
block 11
block 15
P(0-3)
P(4-7)
P(8-11)
P(12-15)
(e) RAID 4 (block-level parity)

  - RAID 5 (block-level distributed parity)


block 12
P(16-19)
P(12-15)
block 16
block 13
block 17
block 14
block 18
block 15
block 19
(f) RAID 5 (block-level distributed parity)

  - RAID 6 (dual redundancy)


block 0
block 4
block 8
block 12
block 1
block 5
block 9
P(12-15)
block 2
block 6
P(8-11)
Q(12-15)
block 3
P(4-7)
Q(8-11)
block 13
P(0-3)
Q(4-7)
block 10
block 14
Q(0-3)
block 7
block 11
block 15
(g) RAID 6 (dual redundancy)

# OS

## CHAPTER 9

- Uniprocessor Scheduling
  - Aim of Scheduling
    - Assign processes to be executed by the processor(s)
    - Scheduling Criteria(准则)
      - Response time(响应时间)
      - Throughput (吞吐量)
      - Processor efficiency
  - Types of Scheduling
    - long-term scheduling
      - the decision to add to the pool of processes to be executed
      - Determines which programs are admitted to the system for processing
      - Controls the degree of multiprogramming
      - More processes, smaller percentage of time each process is executed
    - medium-term scheduling
      - the decision to add to the number of processes that are partially or fully in main memory
      - Part of the swapping function
      - Based on the need to manage the degree of multiprogramming
    - short-term scheduling
      - the decision as to which available process will be executed by the processor
      - Known as the dispatcher
      - Executes most frequently
      - Invoked when an event occurs
        - Clock interrupts
        - I/O interrupts
        - Operating system calls
        - Signals
      - Short-Tem Scheduling Criteria
        - User-oriented
          - Response Time
            - Elapsed time between the submission of a request until there is output.
        - System-oriented
          - Effective and efficient utilization of the processor
        - Performance-related
          - Quantitative
          - Measurable such as response time and throughput

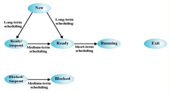

New
Long-term scheduling
Long-term scheduling
Ready/ Suspend
Medium-term scheduling
Ready
Short-term scheduling
Running
Exit
Blocked/ Suspend
Medium-term scheduling
Blocked

Figure 9.1 Scheduling and Process State Transitions

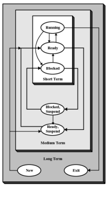

Running
Ready
Blocked
Short Term
Blocked, Suspend
Ready, Suspend
Medium Term
Long Term
New
Exit

Figure 9.2 Levels of Scheduling

  - Queues for Scheduling

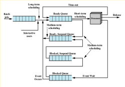

Long-term scheduling
Time-out
Batch jobs
Ready Queue
Short-term scheduling
Processor
Release
Medium-term scheduling
Interactive users
Ready, Suspend Queue
Medium-term scheduling
Blocked, Suspend Queue
Blocked Queue
Event Wait
Event Occurs

Figure 9.3 Queuing Diagram for Scheduling

  - Scheduler will always choose a process of higher priority over one of lower priority
  - Have multiple ready queues to represent each level of priority
  - Lower-priority may suffer starvation(饥饿)
    - Allow a process to change its priority based on its age or execution history

## CHAPTER 3

- Process Description and Control
  - Queues to Implement Five-State Process Model

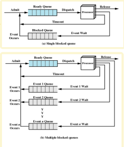

Admit
Ready Queue
Dispatch
Processor
Release
Timeout
Blocked Queue
Event Wait
Event Occurs
(a) Single blocked queue
Admit
Ready Queue
Dispatch
Processor
Release
Timeout
Event 1 Queue
Event 1 Wait
Event 1 Occurs
Event 2 Queue
Event 2 Wait
Event 2 Occurs
Event n Queue
Event n Wait
Event n Occurs
(b) Multiple blocked queues

Figure 3.8 Queuing Model for Figure 3.6

  - Reasons for Process Suspension

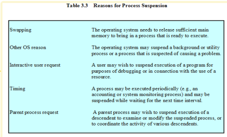


Table 3.3 Reasons for Process Suspension

| | |
|---|---|
| Swapping | The operating system needs to release sufficient main memory to bring in a process that is ready to execute. |
| Other OS reason | The operating system may suspend a background or utility process or a process that is suspected of causing a problem. |
| Interactive user request | A user may wish to suspend execution of a program for purposes of debugging or in connection with the use of a resource. |
| Timing | A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time interval. |
| Parent process request | A parent process may wish to suspend execution of a descendent to examine or modify the suspended process, or to coordinate the activity of various descendents. |

  - Process Control Structures
    - Information about the current status of each process and resource
    - Operating system manages the tables that are constructed for each entity

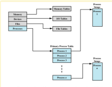

Memory
Devices
Files
Processes
Memory Tables
I/O Tables
File Tables
Process Image
Process 1
Primary Process Table
Process 1
Process 2
Process 3
Process n
Process Image
Process n

Figure 3.11 General Structure of Operating System Control Tables

    - Memory Tables
      - Allocation of main memory to processes
      - Allocation of secondary memory to processes
      - Protection attributes for access to shared memory regions
      - Information needed to manage virtual memory
    - I/O Tables
      - I/O device is available or assigned
      - Status of I/O operation
      - Location in main memory being used as the source or destination of the I/O transfer
    - File Tables
      - Existence of files
      - Location on secondary memory
      - Current Status
      - Attributes
      - Sometimes this information is maintained by a file management system
    - Process Table
      - Where process is located
      - Attributes in the process control block
  - Process Image (进程映像)

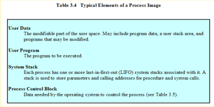


Table 3.4 Typical Elements of a Process Image

**User Data**
The modifiable part of the user space. May include program data, a user stack area, and programs that may be modified.

**User Program**
The program to be executed.

**System Stack**
Each process has one or more last-in-first-out (LIFO) system stacks associated with it. A stack is used to store parameters and calling addresses for procedure and system calls.

**Process Control Block**
Data needed by the operating system to control the process (see Table 3.5).

  - Process Control Block
    - Process identification
      - Identifiers
      - Numeric identifiers that may be stored with the process control block include
    - Processor State Information
      - User-Visible Registers
      - Control and Status Registers
      - Stack Pointers

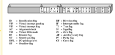


Figure 3.12 Pentium II EFLAGS Register

    - Process Control Information
      - Scheduling and State Information
      - Data Structuring
      - Interprocess Communication
      - Process Privileges
      - Memory Management
      - Resource Ownership and Utilization
  - Process Creation
    - Assign a unique process identifier
    - Allocate space for the process
    - Initialize process control block
    - Set up appropriate linkages
    - Create of expand other data structures
  - When to Switch a Process
    - Clock interrupt
      - process has executed for the maximum allowable time slice
    - Supervisor call
      - such as file open
    - Trap
      - error or exception occurred
      - may cause process to be moved to Exit state
    - I/O interrupt
    - Memory fault
      - memory address is in virtual memory so it must be brought into main memory
  - Change of Process State
    - Save context of processor including program counter and other registers
    - Update the process control block of the process that is currently in the Running state
    - Move process control block to appropriate queue – ready; blocked; ready/suspend
    - Select another process for execution
    - Update the process control block of the process selected
    - Update memory-management data structures
    - Restore context of the selected process
  - Execution of the Operating System
    - Non-process Kernel
      - Execute kernel outside of any process
      - Operating system code is executed as a separate entity that operates in privileged mode
    - Execution Within User Processes
      - Operating system software within context of a user process
      - Process executes in privileged mode when executing operating system code
    - Process-Based Operating System
      - Implement operating system as a collection of system processes
      - Useful in multi-processor or multi-computer environment
  - Relationship Between OS and User Processes

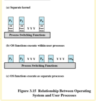

P1
P2
Pn
Kernel
(a) Separate kernel
OS Func-tions
Process Switching Functions
(b) OS functions execute within user processes
OS1
OSk
Process Switching Functions
(c) OS functions execute as separate processes

Figure 3.15 Relationship Between Operating System and User Processes

图 (a)：Separate kernel（独立内核模式）
架构特点：
操作系统的核心（Kernel）是一个 独立的特权层，运行在用户进程（$P_1$、$P_2$…$P_n$）之外。用户进程通过系统调用（图中未显式画，但逻辑上是"陷入内核"）与 Kernel 交互，Kernel 负责管理硬件、资源分配、进程调度等核心功能。
关键逻辑：
Kernel 是 "独立的特权空间"，用户进程不能直接访问硬件或核心资源，必须通过 Kernel 提供的接口（如系统调用）间接操作。
图 (b)：OS functions execute within user processes（OS 功能在用户进程内执行）
架构特点：
操作系统的功能（OS Functions）嵌入到每个用户进程 内部。当用户进程需要 OS 服务（如文件操作、进程切换）时，直接调用自身地址空间内的 OS 逻辑，无需陷入独立的 Kernel。

图 (c)：OS functions execute as separate processes（OS 功能作为独立进程执行）
架构特点：
操作系统的功能被拆分为 独立的 OS 进程（$OS_1$、$OS_2$…$OS_k$），与用户进程（$P_1$、$P_2$…$P_n$）平级，共同依赖 "Process Switching Functions"（进程切换功能，类似调度器）管理。

  - UNIX SVR4 Process States
    - User Running
      - Executing in user mode
    - Kernel Running
      - Executing in kernel mode
    - Ready to Run, in Memory (内存中就绪)
      - Ready to run as soon as the kernel schedules it
    - Asleep in Memory(内存中睡眠)
      - Unable to execute until an event occurs; process is in main memory
    - Ready to Run, Swapped(磁盘上就绪)
      - Process is ready to run, but the swapper must swap the process into main memory before the kernel can schedule it to execute
    - Sleeping, swapped (磁盘上睡眠)
      - The process is awaiting an event and has been swapped to secondary storage
    - Preempted (被抢占)
      - Process is returning from kernel to user mode, but the kernel preempts (抢占) it and does a process switch to schedule another process
    - Created
      - Process is newly created and not yet ready to run
    - Zombie(僵尸)
      - Process no longer exists, but it leaves a record for its parent process to collect

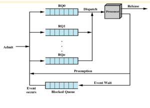

RQ0
Dispatch
Processor
Release
RQ1
RQn
Admit
Preemption
Event Wait
Event occurs
Blocked Queue

Figure 9.4 Priority Queuing

- Priorities
- Decision Mode
  - Nonpreemptive
    - Once a process is in the running state, it will continue until it terminates or blocks itself for I/O
  - Preemptive
    - Currently running process may be interrupted and moved to the Ready state by the operating system
    - Allows for better service since any one process cannot monopolize the processor for very long
- 6 method Process Scheduling

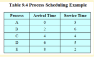
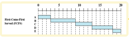


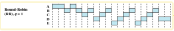
Table 9.4 Process Scheduling Example

| Process | Arrival Time | Service Time |
|---|---|---|
| A | 0 | 3 |
| B | 2 | 6 |
| C | 4 | 4 |
| D | 6 | 5 |
| E | 8 | 2 |

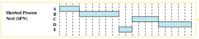
- First-Come-First-Served (FCFS)



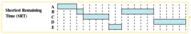
  - Each process joins the Ready queue
  - When the current process ceases to execute, the oldest process in the Ready queue is selected
  - A short process may have to wait a very long time before it can execute
  - Favors CPU-bound processes
    - I/O processes have to wait until CPU-bound process completes
- Round-Robin (RR)



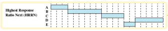
  - Uses preemption based on a clock
  - An amount of time is determined that allows each process to use the processor for that length of time
  - Clock interrupt is generated at periodic intervals
  - When an interrupt occurs, the currently running process is placed in the read queue
    - Next ready job is selected
  - Known as time slicing
- Shortest Process Next



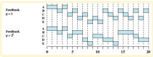
  - Nonpreemptive policy
  - Process with shortest expected processing time is selected next
  - Short process jumps ahead of longer processes
  - Predictability of longer processes is reduced
  - If estimated time for process not correct, the operating system may abort it
  - Possibility of starvation for longer processes
- Shortest Remaining Time



  - Preemptive version of shortest process next policy
  - Must estimate processing time
- Highest Response Ratio Next (HRRN)



  - Choose next process with the greatest ratio
  - (time spent waiting + expected service time) /expected service time
- Feedback



  - Penalize jobs that have been running longer
  - Don't know remaining time process needs to execute

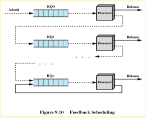

Admit
RQ0
Processor
Release
RQ1
Processor
Release
RQn
Processor
Release

Figure 9.10 Feedback Scheduling

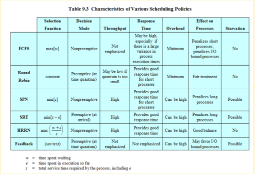


Table 9.3 Characteristics of Various Scheduling Policies

| | Selection Function | Decision Mode | Throughput | Response Time | Overhead | Effect on Processes | Starvation |
|---|---|---|---|---|---|---|---|
| **FCFS** | max[$w$] | Nonpreemptive | Not emphasized | May be high, especially if there is a large variance in process execution times | Minimum | Penalizes short processes; penalizes I/O bound processes | No |
| **Round Robin** | constant | Preemptive (at time quantum) | May be low if quantum is too small | Provides good response time for short processes | Minimum | Fair treatment | No |
| **SPN** | min[$s$] | Nonpreemptive | High | Provides good response time for short processes | Can be high | Penalizes long processes | Possible |
| **SRT** | min[$s - e$] | Preemptive (at arrival) | High | Provides good response time | Can be high | Penalizes long processes | Possible |
| **HRRN** | $\max\left(\frac{w+s}{s}\right)$ | Nonpreemptive | High | Provides good response time | Can be high | Good balance | No |
| **Feedback** | (see text) | Preemptive (at time quantum) | Not emphasized | Not emphasized | Can be high | May favor I/O bound processes | Possible |

$w$ = time spent waiting
$e$ = time spent in execution so far
$s$ = total service time required by the process, including $e$

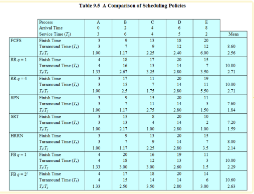
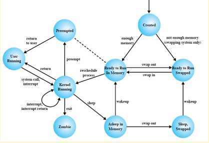


Table 9.5 A Comparison of Scheduling Policies

| | Process | A | B | C | D | E | |
|---|---|---|---|---|---|---|---|
| | Arrival Time | 0 | 2 | 4 | 6 | 8 | |
| | Service Time ($T_s$) | 3 | 6 | 4 | 5 | 2 | Mean |
| FCFS | Finish Time | 3 | 9 | 13 | 18 | 20 | |
| | Turnaround Time ($T_r$) | 3 | 7 | 9 | 12 | 12 | 8.60 |
| | $T_r/T_s$ | 1.00 | 1.17 | 2.25 | 2.40 | 6.00 | 2.56 |
| RR $q = 1$ | Finish Time | 4 | 18 | 17 | 20 | 15 | |
| | Turnaround Time ($T_r$) | 4 | 16 | 13 | 14 | 7 | 10.80 |
| | $T_r/T_s$ | 1.33 | 2.67 | 3.25 | 2.80 | 3.50 | 2.71 |
| RR $q = 4$ | Finish Time | 3 | 17 | 11 | 20 | 19 | |
| | Turnaround Time ($T_r$) | 3 | 15 | 7 | 14 | 11 | 10.00 |
| | $T_r/T_s$ | 1.00 | 2.5 | 1.75 | 2.80 | 5.50 | 2.71 |
| SPN | Finish Time | 3 | 9 | 15 | 20 | 11 | |
| | Turnaround Time ($T_r$) | 3 | 7 | 11 | 14 | 3 | 7.60 |
| | $T_r/T_s$ | 1.00 | 1.17 | 2.75 | 2.80 | 1.50 | 1.84 |
| SRT | Finish Time | 3 | 15 | 8 | 20 | 10 | |
| | Turnaround Time ($T_r$) | 3 | 13 | 4 | 14 | 2 | 7.20 |
| | $T_r/T_s$ | 1.00 | 2.17 | 1.00 | 2.80 | 1.00 | 1.59 |
| HRRN | Finish Time | 3 | 9 | 13 | 20 | 15 | |
| | Turnaround Time ($T_r$) | 3 | 7 | 9 | 14 | 7 | 8.00 |
| | $T_r/T_s$ | 1.00 | 1.17 | 2.25 | 2.80 | 3.5 | 2.14 |
| FB $q = 1$ | Finish Time | 4 | 20 | 16 | 19 | 11 | |
| | Turnaround Time ($T_r$) | 4 | 18 | 12 | 13 | 3 | 10.00 |
| | $T_r/T_s$ | 1.33 | 3.00 | 3.00 | 2.60 | 1.5 | 2.29 |
| FB $q = 2^i$ | Finish Time | 4 | 17 | 18 | 20 | 14 | |
| | Turnaround Time ($T_r$) | 4 | 15 | 14 | 14 | 6 | 10.60 |
| | $T_r/T_s$ | 1.33 | 2.50 | 3.50 | 2.80 | 3.00 | 2.63 |

- Basic Idea of Virtual Memory Management
  - A process may be swapped in and out of main memory such that it occupies different regions.
  - Memory references are dynamically translated into physical addresses at run time
  - A process may be broken up into pieces that do not need to located contiguously(连续的) in main memory
  - All pieces of a process do not need to be loaded in main memory during execution
- Execution of a Program
  - Operating system only brings into main memory a few pieces of the program
  - Resident set(驻留集) - portion of process that is in main memory
  - An interrupt(Page Fault, 页面故障) is generated when an address is needed that is not in main memory. Operating system places the process in blocking state
  - Piece of process that contains the logical address is brought into main memory
    - Operating system issues a disk I/O Read request
    - Another process is dispatched to run while the disk I/O takes place
    - An interrupt is issued when disk I/O complete which causes the operating system to place the affected process in the Ready state
- Principle of Locality(局部性原理)
  - Program and data references within a process tend to cluster
  - Only a few pieces of a process will be needed over a short period of time
  - Possible to make intelligent guesses about which pieces will be needed in the future
  - This suggests that virtual memory may work efficiently


Created
enough memory
not enough memory (swapping system only)
Preempted
return to user
User Running
preempt
return
system call, interrupt
Kernel Running
reschedule process
Ready to Run In Memory
swap out
swap in
Ready to Run Swapped
interrupt, interrupt return
exit
sleep
wakeup
wakeup
Zombie
Asleep in Memory
swap out
Sleep, Swapped

## CHAPTER 4

- Threads, SMP and Microkernels
  - Process
    - Scheduling/execution- follows an execution path that may be interleaved with other processes
    - Resource ownership - process includes a virtual address space to hold the process image
  - Multithreading
    - Modern operating system supports multiple threads of execution within a single process

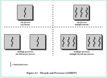

one process one thread
one process multiple threads
multiple processes one thread per process
multiple processes multiple threads per process
= instruction trace

Figure 4.1 Threads and Processes [ANDE97]

    - In a multithreaded environment, a process is defined as the unit of resource allocation and a unit of protection:
      - Have a virtual address space which holds the process image
      - Protected access to processors, other processes (for interprocess communication), files, and I/O resources (devices and channels)
    - Within a process, there may be one or more threads, each with the following:
      - An execution state (running, ready, etc.)
      - Saved thread context when not running
      - Has an execution stack
      - Some per-thread static storage for local variables
      - Access to the memory and resources of its process
    - Single Threaded and Multithreaded Process Models

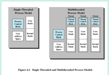
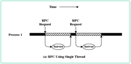
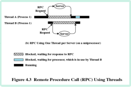

Single-Threaded Process Model
Process Control Block
User Stack
User Address Space
Kernel Stack
Multithreaded Process Model
Thread
Thread Control Block
User Stack
Kernel Stack

Figure 4.2 Single Threaded and Multithreaded Process Models

  - Benefits of Multithreading
    - Takes less time to create a new thread than a process
    - Less time to terminate a thread than a process
    - Less time to switch between two threads within the same process
    - Since threads within the same process share memory and files, they can communicate with each other without invoking the kernel
  - Multithreaded Scenarios
    - Foreground to background work
    - Asynchronous processing
    - Speed of execution
    - Modular program structure
  - Thread States
    - States associated with a change in thread state
      - Spawn
        - Spawn another thread
      - Block
      - Unblock
      - Finish
        - Deallocate register context and stacks
  - Multithreading Example - RPC
    - A program that performs two remote procedure calls (RPCs) to two different hosts to obtain a combined result
    - RPCs use single thread
    - RPCs use multiple threads


Time
RPC Request
RPC Request
Process 1
Server
Server

(a) RPC Using Single Thread


RPC Request
Server
Thread A (Process 1)
Thread B (Process 1)
RPC Request
Server
Blocked, waiting for response to RPC
Blocked, waiting for processor, which is in use by Thread B
Running

(b) RPC Using One Thread per Server (on a uniprocessor)

Figure 4.3 Remote Procedure Call (RPC) Using Threads

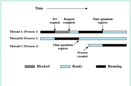
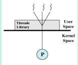
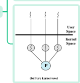

Time
I/O request
Request complete
Time quantum expires
Thread A (Process 1)
Thread B (Process 1)
Thread C (Process 2)
Time quantum expires
Process created
Blocked
Ready
Running

Figure 4.4 Multithreading Example on a Uniprocessor

  - Different Threading Implementations
    - User-level threads (用户级多线程)
      - All thread management is done by the application
      - The kernel is not aware of the existence of threads

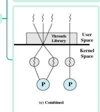

Threads Library
User Space
Kernel Space
P

    - Kernel-level threads (核心级多线程)
      - Kernel maintains context information for the process and the threads
        - Windows is an example of this approach
      - Scheduling is done on a thread basis


User Space
Kernel Space
P

(b) Pure kernel-level

    - Combined Approaches
      - Thread creation done in the user space
      - Bulk of scheduling and synchronization of threads within application

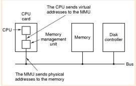
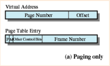
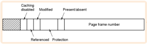
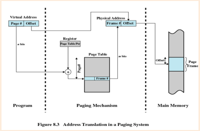
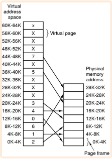
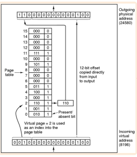

Threads Library
User Space
Kernel Space
P
P

(c) Combined

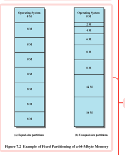
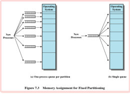
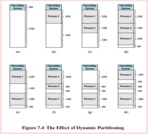
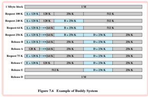
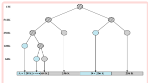
  - Symmetric Multiprocessing(对称多处理)
    - Kernel can execute on any processor
    - Typically each processor does self-scheduling from the pool of available process or threads
  - Microkernels (微内核)
    - Small operating system core
    - Contains only essential core operating systems functions
    - Many services traditionally included in the operating system are now external subsystems
    - Device drivers
    - File systems
    - Virtual memory manager
    - Windowing system
    - Security services
  - Benefits of Microkernel Organization
    - Uniform interface on request made by a process
      - Don't distinguish between kernel-level and user-level services
      - All services are provided by means of message passing
    - Extensibility
      - Allows the addition of new services
    - Flexibility
      - New features added
    - Portability
      - Changes needed to port the system to a new processor is changed in the microkernel - not in the other services
    - Reliability
      - Modular design
      - Small microkernel can be rigorously tested

- Distributed system support
  - Message are sent without knowing what the target machine is
- Object-oriented operating system
  - Components are objects with clearly defined interfaces that can be interconnected to form software

## CHAPTER 8

- Virtual Memory
  - Paging
    - 
The CPU sends virtual addresses to the MMU
CPU card
CPU
Memory management unit
Memory
Disk controller
Bus
The MMU sends physical addresses to the memory
    - Each process has its own page table
    - Each page table entry contains the frame number of the corresponding page in main memory.
    - A bit is needed in PTE to indicate whether the page is in main memory or not
    - Virtual Address Space
  - Virtual Address and Page Table Entry
    - 
Virtual Address
Page Number
Offset
Page Table Entry
P
M
Other Control Bits
Frame Number
(a) Paging only
      - 
Caching disabled
Modified
Present/absent
Page frame number
Referenced
Protection
      - Modify Bit in Page Table
        - Modify bit is needed to indicate if the page has been altered since it was last loaded into main memory
        - If no change has been made, the page does not have to be written to the disk when it needs to be swapped out
  - Address Translation(Mapping)
    - 
Virtual Address
Page #
Offset
Register
Page Table Ptr
Page Table
Frame #
Page#
Physical Address
Frame #
Offset
Page Frame
Program
Paging Mechanism
Main Memory
Figure 8.3 Address Translation in a Paging System
      - 
Virtual address space
60K-64K
56K-60K
52K-56K
48K-52K
44K-48K
40K-44K
36K-40K
32K-36K
28K-32K
24K-28K
20K-24K
16K-20K
12K-16K
8K-12K
4K-8K
0K-4K
Virtual page
Physical memory address
Page frame
        - 
Outgoing physical address (24580)
Page table
12-bit offset copied directly from input to output
Present/ absent bit
Virtual page = 2 is used as an index into the page table
Incoming virtual address (8196)
110
  - Page Tables
    - The entire page table may take up too much main memory
    - Page tables are also stored in virtual memory
    - When a process is running, part of its page table is in main memory

## CHAPTER 7

- Memory Management
  - Basic Idea of Memory Management
    - Subdividing memory to accommodate multiple processes
    - Memory needs to be allocated to ensure a reasonable supply(合理供应) of ready processes to consume available processor time
  - Memory Management Requirements
    - Relocation(重定位)
      - Programmer does not know where the program will be placed in memory when it is executed
      - While the program is executing, it may be swapped to disk and returned to main memory at a different location (relocated)
      - Memory references must be translated in the code to actual physical memory address
    - Protection(保护)
      - Processes should not be able to reference memory locations in another process without permission
      - Impossible to check absolute addresses at compile time
      - Address reference must be checked at run time
      - Memory protection requirement must be satisfied by the processor (hardware) rather than the operating system (software)
        - Operating system cannot anticipate all of the memory references a program will make
    - Sharing(共享)
      - Allow several processes to access the same portion of memory
        - Better to allow each process access to the same copy of the program rather than have their own separate copy
    - Logical Organization
      - Programs are written in modules
      - Modules can be written and compiled independently
      - Different degrees of protection given to modules (read-only, execute-only)
      - Share modules among processes
    - Physical Organization
      - Memory available for a program plus its data may be insufficient
        - Overlaying allows various modules to be assigned the same region of memory
      - Programmer does not know how much space will be available
  - Fixed Partitioning
    - Equal-size partitions
      - Any process whose size is less than or equal to the partition size can be loaded into an available partition
      - If all partitions are full, the operating system can swap a process out of a partition
      - A program may not fit in a partition. The programmer must design the program with overlays(覆盖)
    - Main memory use is inefficient. Any program, no matter how small, occupies an entire partition. This is called internal fragmentation(内零头/内部碎片).
    - 
Operating System 8 M
8 M
2 M
4 M
6 M
12 M
16 M
(a) Equal-size partitions
(b) Unequal-size partitions
Figure 7.2 Example of Fixed Partitioning of a 64-Mbyte Memory
    - Placement Algorithm with Partitions
      - Equal-size partitions
        - Because all partitions are of equal size, it does not matter which partition is used
      - Unequal-size partitions
        - Can assign each process to the smallest partition within which it will fit
        - Queue for each partition
        - Processes are assigned in such a way as to minimize wasted memory within a partition
    - Memory Assignment for Fixed Partitioning
      - 
Operating System
New Processes
(a) One process queue per partition
(b) Single queue
Figure 7.3 Memory Assignment for Fixed Partitioning
  - Dynamic Partitioning
    - Partitions are of variable length and number
    - Process is allocated exactly as much memory as required
    - Eventually get holes in the memory. This is called external fragmentation (外零头/外部碎片)
    - Must use compaction(紧缩) to shift processes so they are contiguous and all free memory is in one block
    - The Effect of Dynamic Partitioning
      - 
Operating System
Process 1
Process 2
Process 3
Process 4
8M
56M
20M
36M
14M
22M
18M
4M
6M
(a)
(b)
(c)
(d)
(e)
(f)
(g)
(h)
Figure 7.4 The Effect of Dynamic Partitioning
    - Dynamic Partitioning Placement Algorithm
      - Best-fit(最佳适配) algorithm
        - Chooses the block that is closest in size to the request
        - Worst performer overall
        - Since smallest block is found for process, the smallest amount of fragmentation is left
        - Memory compaction must be done more often
      - First-fit(首次适配) algorithm
        - Scans memory from the beginning and chooses the first available block that is large enough
        - May have many process loaded in the front end of memory that must be searched over when trying to find a free block
      - Next-fit(下次适配)
        - Scans memory from the location of the last placement
        - More often allocate a block of memory at the end of memory where the largest block is found
        - The largest block of memory is broken up into smaller blocks
        - Compaction is required to obtain a large block at the end of memory
    - Buddy System
      - Entire space available is treated as a single block of 2U
      - If a request of size s such that 2U-1 < s <= 2U, entire block is allocated
        - Otherwise block is split into two equal buddies
        - Process continues until smallest block greater than or equal to s is generated
      - 
1 Mbyte block
Request 100 K
Request 240 K
Request 64 K
Request 256 K
Release B
Release A
Request 75 K
Release C
Release E
Release D
1 M
A = 128 K
B = 256 K
C = 64 K
D = 256 K
E = 128 K
128 K
256 K
512 K
64 K
Figure 7.6 Example of Buddy System
      - 
1M
512K
256K
128K
64K
A = 128 K
C = 64 K
64 K
256 K
D = 256 K
256 K

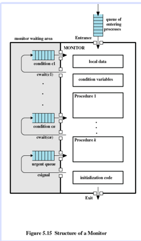
## CHAPTER 5

- Concurrency: Mutual Exclusion and Synchronization
  - Concurrency Overview
    - The central themes of operating system design are all concerned with the management of processes and threads:
      - Multiprogramming
      - Multiprocessing
      - Distributed processing
    - Fundamental to operating system design is concurrency(并发)
    - The Basic requirement for support of concurrency is the ability to enforce mutual exclusion(互斥)
  - Concurrency Contexts(并发上下文)
    - Concurrency arises in three different contexts:
      - Multiple applications
        - Multiprogramming
      - Structured application
        - Application can be a set of concurrent processes or threads
      - Operating-system structure
        - Operating system is a set of processes or threads
  - Principles of Concurrency
    - Interleaving
    - Overlapping
    - Basic characteristic of multiprogramming systems: relative speed of execution of processes cannot be predicted
  - Race Condition(竞争条件)
    - When multiple processes or threads access shared resource so that the final result depends on the order of execution of instructions in the multiple processes
    - Murphy Law
      - If Something can go wrong, it will.
    - Solution to Race Condition:
      - Control access to the shared resource
    - Mutual Exclusion(互斥)
  - Process Interaction
    - Three possible degrees of awareness among processes:
      - Processes unaware of each other
        - Competition
      - Processes indirectly aware of each other
        - Cooperation
      - Process directly aware of each other
        - Cooperation
  - Competition Among Processes for Resources
    - Critical sections(临界区)
      - Portion of the program that access shared resource
        - Only one program at a time is allowed in its critical section
    - Enforcement of mutual exclusion incurs two additional problems:
      - Deadlock(死锁)
      - Starvation(饥饿)
  - Requirements for Mutual Exclusion
    - Only one process at a time is allowed in the critical section for a resource
    - A process that halts in its noncritical section(非临界区) must do so without interfering with other processes
    - No deadlock or starvation
    - A process must not be delayed access to a critical section when there is no other process using it
    - No assumptions are made about relative process speeds or number of processes
    - A process remains inside its critical section for a finite time only
  - Mutual Exclusion:Hardware Support
    - Interrupt Disabling(关中断)
      - A process runs until it invokes an operating system service or until it is interrupted
      - Disabling interrupts guarantees mutual exclusion
      - Processor is limited in its ability to interleave programs
      - Multiprocessing
        - disabling interrupts on one processor will not guarantee mutual exclusion
    - Special Machine Instructions
      - Performed in a single instruction cycle
      - Access to the memory location is blocked for any other instructions
    - Test and Set Instruction
    - Exchange Instruction
  - Semaphores(信号量)
    - Special variable called a semaphore is used for signaling
    - Two primitives(原语) are defined:
      - Wait operation decrements the semaphore value
      - Signal operation increments semaphore value
      - P/V operations
      - Down/Up operations
    - Two kinds of semaphores:
      - counting semaphores: can take on any non-negative value
      - binary semaphores: take on only the values 0 and 1.(Von a binary semaphore with value 1 has no effect.)
  - Producer/Consumer Problem
    - One or more producers are generating data and placing these in a buffer
    - A single consumer is taking items out of the buffer one at time
    - Only one producer or consumer may access the buffer at any one time

```
#define  N 100
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;
```

```
void producer(void)
{
   int item;
   while(true){
       produce_item(&Item);
       semWait(&empty);
       semWait(&mutex);
       enter_item(item);
       semSignal(&mutex);
       semSignal(&full);
   }
}
```

```
void consumer(void)
{
   int item;
   while(true){
       semWait(&full);
       semWait(&mutex);
       remove_item(&item);
       semSignal(&mutex);
       semSignal(&empty)
       consume_item(item);
   }
}
```

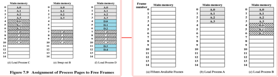
  - Monitors(管程)
    - Monitor is a software module consisting of one or more procedures, an initialization sequence, and local data
    - Chief characteristics
      - Local data variables are accessible only by the monitor's procedures and not by any external procedure
      - Process enters monitor by invoking one of its procedures
      - Only one process may be executing in the monitor at a time
    - Condition Variables
      - Condition variable are a special data type in monitors, which are operated on by two operations:
        - cwait(c): suspend execution of the calling process on condition c. The monitor is now available for use by another process
        - csignal(c): resume execution of some process blocked after a cwait on the same condition. If there are several such processes, choose one of them; if there is no such process, do nothing.
    - 
queue of entering processes
monitor waiting area
Entrance
MONITOR
condition c1
cwait(c1)
condition cn
cwait(cn)
urgent queue
csignal
local data
condition variables
Procedure 1
Procedure k
initialization code
Exit
Figure 5.15 Structure of a Monitor
  - Message Passing(消息传递)
    - Enforce mutual exclusion
    - Exchange information
    - A pair of primitives is provided for message passing:
      - send(destination, message)
      - receive(source, message)
  - Synchronization（同步）
    - Sender and receiver may or may not be blocking (waiting for message)
    - Blocking send, blocking receive
      - Both sender and receiver are blocked until message is delivered
      - Called a rendezvous (会合)
    - Nonblocking send, blocking receive
      - Sender continues on
      - Receiver is blocked until the requested message arrives
    - Nonblocking send, nonblocking receive
      - Neither party is required to wait
  - Addressing
    - Direct addressing
      - Send primitive includes a specific identifier of the destination process
      - Receive primitive could know ahead of time which process a message is expected
      - Receive primitive could use source parameter to return a value when the receive operation has been performed
    - Indirect addressing
      - Messages are sent to a shared data structure consisting of queues
      - Queues are called mailboxes
      - One process sends a message to the mailbox and the other process picks up the message from the mailbox
  - Readers/Writers Problem
    - Any number of readers may simultaneously read the file
    - Only one writer at a time may write to the file
    - If a writer is writing to the file, no reader may read it

```
/* program readersandwriters */
int readcount;
semaphore x = 1, wsem = 1;
void reader()
{
   while (true)
   {
    semWait (x);
    readcount++;
    if (readcount == 1)
        semWait (wsem);
    semSignal (x);
    READUNIT();
    semWait (x);
    readcount--;
    if (readcount == 0)
        semSignal (wsem);
    semSignal (x);
   }
 }
void writer()
{
   while (true)
   {
    semWait (wsem);
    WRITEUNIT();
    semSignal (wsem);
   }
}

void main()
{
   readcount = 0;
   parbegin (reader, writer);
   }
```

Figure 7.7 Tree Representation of Buddy System

- Addresses
  - Logical
    - Reference to a memory location independent of the current assignment of data to memory
    - Translation must be made to the physical address
  - Relative
    - Address expressed as a location relative to some known point
  - Physical
    - The absolute address or actual location in main memory
- Paging(分页)
  - Partition memory into small equal fixed-size chunks and divide each process into the same size chunks
  - The chunks of a process are called pages(页) and chunks of memory are called frames(页框)
  - Operating system maintains a page table for each process
    - Contains the frame location for each page in the process
    - Memory address consist of a page number and offset within the page


Frame number
Main memory
A.0
A.1
A.2
A.3
B.0
B.1
B.2
C.0
C.1
C.2
C.3
D.0
D.1
D.2
D.3
D.4

(a) Fifteen Available Frames (b) Load Process A (c) Load Process B (d) Load Process C (e) Swap out B (f) Load Process D

Figure 7.9 Assignment of Process Pages to Free Frames

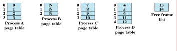
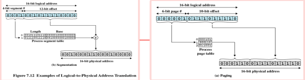

Process A page table
Process B page table
Process C page table
Process D page table
Free frame list

Figure 7.10 Data Structures for the Example of Figure 7.9 at Time Epoch (f)

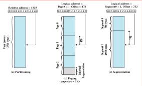

16-bit logical address
4-bit segment #
12-bit offset
Length
Base
Process segment table
16-bit physical address
6-bit page #
10-bit offset
Process page table

(b) Segmentation

(a) Paging

Figure 7.12 Examples of Logical-to-Physical Address Translation

- Segmentation(分段)
  - All segments of all programs do not have to be of the same length
  - There is a maximum segment length
  - Addressing consist of two parts - a segment number and an offset
  - Since segments are not equal, segmentation is similar to dynamic partitioning
- Logical Addresses

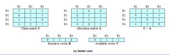

Relative address = 1502
Logical address = Page# = 1, Offset = 478
Logical address = Segment# = 1, Offset = 752
User process (2700 bytes)
Page 0
Page 1
Page 2
Internal fragmentation
Segment 0 750 bytes
Segment 1 1950 bytes
478
752

(a) Partitioning (b) Paging (page size = 1K) (c) Segmentation

Figure 7.11 Logical Addresses

Figure 5.22 A Solution to the Readers/Writers Problem Using Semaphores: Readers Have Priority

- CHAPTER 6
  - Concurrency: Deadlock and Starvation
    - Deadlock (死锁)
      - Permanent blocking(永久阻塞) of a set of processes that either compete for system resources or communicate with each other
      - No efficient solution
        - Involve conflicting needs for resources by two or more processes
      - Deadlock occurs if receive is blocking
      - May take a rare combination of events to cause deadlock
      - Conditions for Deadlock
        - Mutual exclusion
          - Only one process may use a resource at a time (一个资源在某个时间只能被一个进程使用)
        - Hold-and-wait
          - A process may hold allocated resources while awaiting assignment of others
        - No preemption
          - No resource can be forcibly removed from a process holding it
        - Circular wait
          - A closed chain of processes exists, such that each process holds at least one resource needed by the next process in the chain
      - Possibility and Existence of Deadlock
        - Possibility of Deadlock
          - Mutual Exclusion
          - No preemption
          - Hold and wait
        - Existence of Deadlock
          - Mutual Exclusion
          - No preemption
          - Hold and wait
          - Circular wait
        - Necessary Conditions
        - Sufficient Conditions
      - Deadlock Prevention (死锁预防)
        - The strategy of deadlock prevention is to design a system in such a way that the possibility of deadlock is excluded
        - Indirect method
          - prevent the occurrence of one of the three necessary conditions
        - Direct method
          - prevent the occurrence of a circular wait
        - Mutual Exclusion
          - Spooling
          - File read and write (must be supported by the operating system)
        - Hold and Wait
          - Require a process request all of its required resources at one time and blocking the process until all requests can be granted simultaneously
          - Inefficient
          - impractical
        - No Preemption
          - Process must release resource and request again
          - Operating system may preempt a process to require it releases its resources
          - Practical only when applied to resources whose state can be easily saved and restored later
        - Circular Wait
          - Define a linear ordering of resource types
        - Constrain resource requests to prevent at least one of the four conditions of deadlock
        - Leads to inefficient use of resources and inefficient execution of processes
      - Deadlock Avoidance (死锁避免)
        - Allows the three necessary conditions but makes judicious choices to assure that the deadlock point is never reached
        - Avoidance allows more concurrency than prevention
        - A decision is made dynamically whether the current resource allocation request will, if granted, potentially lead to a deadlock
        - Requires knowledge of future process request
      - Approaches to Deadlock Avoidance
        - Do not start a process if its demands might lead to deadlock
          - Process initiation denial
        - Do not grant an incremental resource request to a process if this allocation might lead to deadlock
          - Resource allocation denial
      - Deadlock Detection
        - Deadlock prevention strategies are very conservative(保守)
        - Deadlock detection strategies do not limit resource access or restrict process actions
        - Requested resources are granted to processes whenever possible
        - Operating system performs an algorithm that allows it to detect the circular wait condition
    - Reusable Resources (可重用资源)
      - Used by only one process at a time and not depleted(耗尽) by that use
      - Processes obtain resources that they later release for reuse by other processes
      - Deadlock occurs if each process holds one resource and requests the other
      - Possible Reusable Resources: processors, I/O channels, main and secondary memory, devices, and data structures such as files, databases, and semaphores
    - Consumable Resources (可消耗的资源)
      - Can be created (produced) and destroyed (consumed)
      - When a resource is acquired by a consuming process, the resource ceased to exist
      - Examples of consumable resources: interrupts, signals, messages, and information in I/O buffers
    - Resource Allocation Denial
      - Referred to as the banker's algorithm(银行家算法)
      - State of the system is the current allocation of resources to process
      - Safe state is where there is at least one sequence that does not result in deadlock
      - Unsafe state is a state that is not safe



|  | R1 | R2 | R3 |
|---|---|---|---|
| P1 | 3 | 2 | 2 |
| P2 | 6 | 1 | 3 |
| P3 | 3 | 1 | 4 |
| P4 | 4 | 2 | 2 |

Claim matrix C

|  | R1 | R2 | R3 |
|---|---|---|---|
| P1 | 1 | 0 | 0 |
| P2 | 6 | 1 | 2 |
| P3 | 2 | 1 | 1 |
| P4 | 0 | 0 | 2 |

Allocation matrix A

|  | R1 | R2 | R3 |
|---|---|---|---|
| P1 | 2 | 2 | 2 |
| P2 | 0 | 0 | 1 |
| P3 | 1 | 0 | 3 |
| P4 | 4 | 2 | 0 |

C – A

| R1 | R2 | R3 |
|---|---|---|
| 9 | 3 | 6 |

Resource vector R

| R1 | R2 | R3 |
|---|---|---|
| 0 | 1 | 1 |

Available vector V

(a) Initial state

- Restrictions to Deadlock Avoidance
  - Maximum resource requirement must be stated in advance
  - Processes under consideration must be independent; no synchronization requirements
  - There must be a fixed number of resources to allocate
  - No process may exit while holding resources